

Actividad de Investigación y Práctica: Estructuras de Datos Avanzadas y APIs con ASP.NET Core

Modalidad: Individual

Entregable sugerido: Un documento en Markdown (.md) con la investigación y el enlace al repositorio de código o capturas de las pruebas de la API.

Parte 1: Investigación Teórica

El objetivo de esta sección es comprender los fundamentos conceptuales de la organización de datos en memoria y su posterior exposición a través de servicios web. Investiga y responde brevemente a los siguientes puntos:

1. Estructuras de Datos Eficientes

- **Árboles Binarios de Búsqueda (ABB):** Explica su regla de ordenamiento (hijo izquierdo e hijo derecho) y cuál es su principal desventaja cuando los datos se insertan en un orden secuencial (caso de degeneración en lista vinculada).
- **Árboles AVL:** Define qué es un árbol auto-balanceado. Explica brevemente el concepto de *factor de balanceo* ($\text{Factor} = \text{Altura}_{\text{Izquierda}} - \text{Altura}_{\text{Derecha}}$) y por qué su complejidad para operaciones de búsqueda, inserción y eliminación se mantiene siempre en $O(\log n)$.

2. Fundamentos de Web APIs

- **¿Qué es una API y cómo funciona el modelo Cliente-Servidor?** Describe cómo viaja una petición (Request) y una respuesta (Response) a través del protocolo HTTP.
- **Verbos HTTP:** Define la diferencia conceptual, el uso correcto y la propiedad de idempotencia para:
 - GET: Recuperación de recursos.
 - POST: Creación de nuevos recursos.

Parte 2: Implementación Práctica en C# con ASP.NET Core

Para consolidar los conocimientos, desarrollarás una Web API minimalista utilizando **ASP.NET Core (Minimal APIs o Controladores)** en .NET 8 o superior. La API simulará la gestión de un catálogo de nodos o elementos numéricos.

Requisitos previos:

- SDK de .NET (.NET 6, 7 o 8) instalado.
- Un editor de código (VS Code o Visual Studio).
- Una herramienta para probar la API (Postman, Bruno, o la extensión *REST Client*).

Instrucciones de Desarrollo:

1. Creación del proyecto:

Abre tu terminal y ejecuta el siguiente comando para crear una API web limpia:

Bash

```
dotnet new webapi -o ApiEstructurasDemo
```

```
cd ApiEstructurasDemo
```

2. Modelado del Recurso (El "Nodo"):

Crea una clase simple que represente un elemento que podría pertenecer a un árbol (por ejemplo, un identificador numérico y un valor).

C#

```
public class NodoElemento
{
    public int Id { get; set; }
    public string Valor { get; set; } = string.Empty;
}
```

3. Configuración del almacenamiento en memoria:

Para simular una estructura (como la colección de elementos que insertarías en un árbol), utiliza una lista estática en memoria dentro de tu Program.cs o en un controlador.

4. Implementación de Endpoints (GET y POST):

Configura los dos métodos requeridos en el archivo Program.cs:

C#

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```

```
// Base de datos simulada en memoria
```

```
var coleccionNodos = new List<NodoElemento>
```

```
{
    new NodoElemento { Id = 10, Valor = "Raíz Inicial (ABB)" },
    new NodoElemento { Id = 5, Valor = "Hijo Izquierdo" }
};
```

```
// 1. EJEMPLO DE GET: Retorna todos los nodos actuales
```

```
app.MapGet("/api/nodos", () => Results.Ok(coleccionNodos));
```

```
// 2. EJEMPLO DE POST: Recibe un nuevo nodo y lo "inserta" en la colección
```

```
app.MapPost("/api/nodos", (NodoElemento nuevoNodo) =>
```

```
{
```

```
    // Validación simple
```

```
    if (nuevoNodo.Id <= 0 || string.IsNullOrEmpty(nuevoNodo.Valor))
```

```
{
```

```
        return Results.BadRequest("Datos del nodo inválidos.");
    }

    coleccionNodos.Add(nuevoNodo);
    return Results.Created($"{api/nodos/{nuevoNodo.Id}", nuevoNodo);
});

app.Run();
```

5. Ejecución de la API:

Corre el proyecto usando:

Bash

dotnet run

Parte 3: Verificación y Pruebas (Evaluación interna)

Para dar por concluida la hora de actividad, debes probar el comportamiento de tu API utilizando cualquier cliente HTTP (como Postman) realizando las siguientes acciones:

1. Prueba del GET:

- Envía una petición GET a `http://localhost:XXXX/api/nodos` (reemplaza XXXX por tu puerto local).
- **Resultado esperado:** Recibir un código de estado 200 OK junto con el JSON que contiene los dos nodos iniciales.

2. Prueba del POST:

- Envía una petición POST a `http://localhost:XXXX/api/nodos`.
- En el cuerpo (Body) de la petición, selecciona formato JSON y envía un nuevo objeto, simulando una inserción (por ejemplo, un ID que balancearía un árbol AVL):

JSON

```
{
  "id": 15,
  "valor": "Nuevo Nodo Derecho"
}
```

- **Resultado esperado:** Recibir un código 201 Created. Al volver a ejecutar el comando GET, el nuevo elemento debe aparecer listado en la colección.